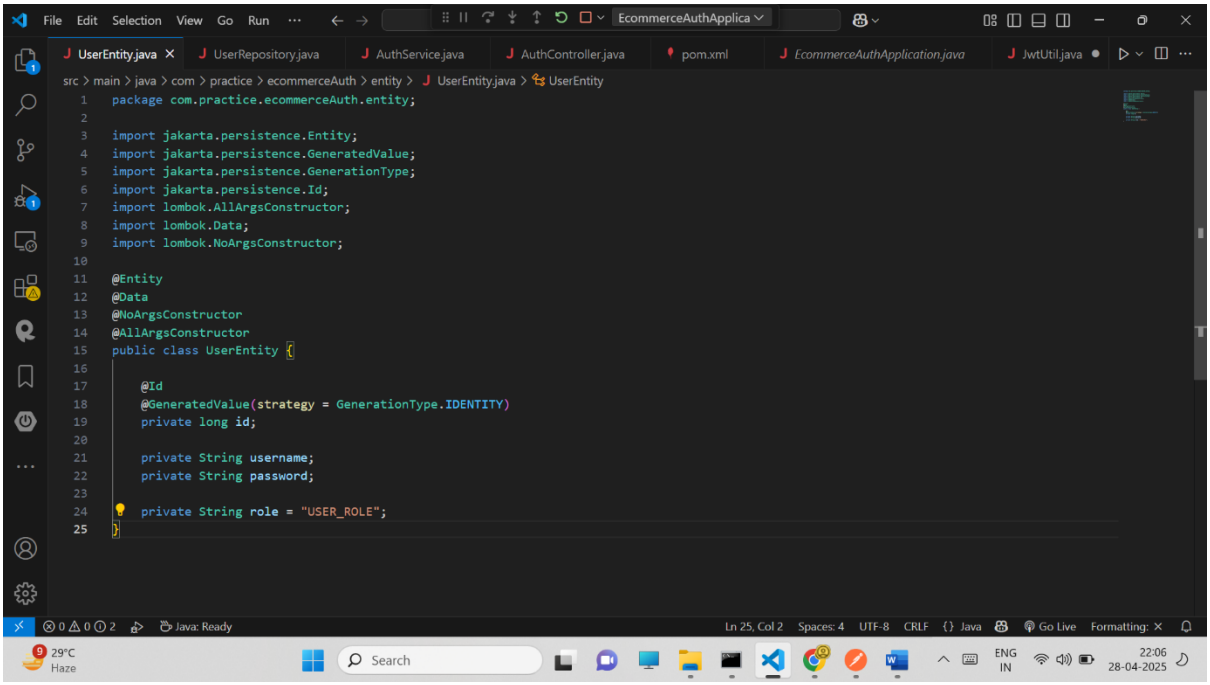
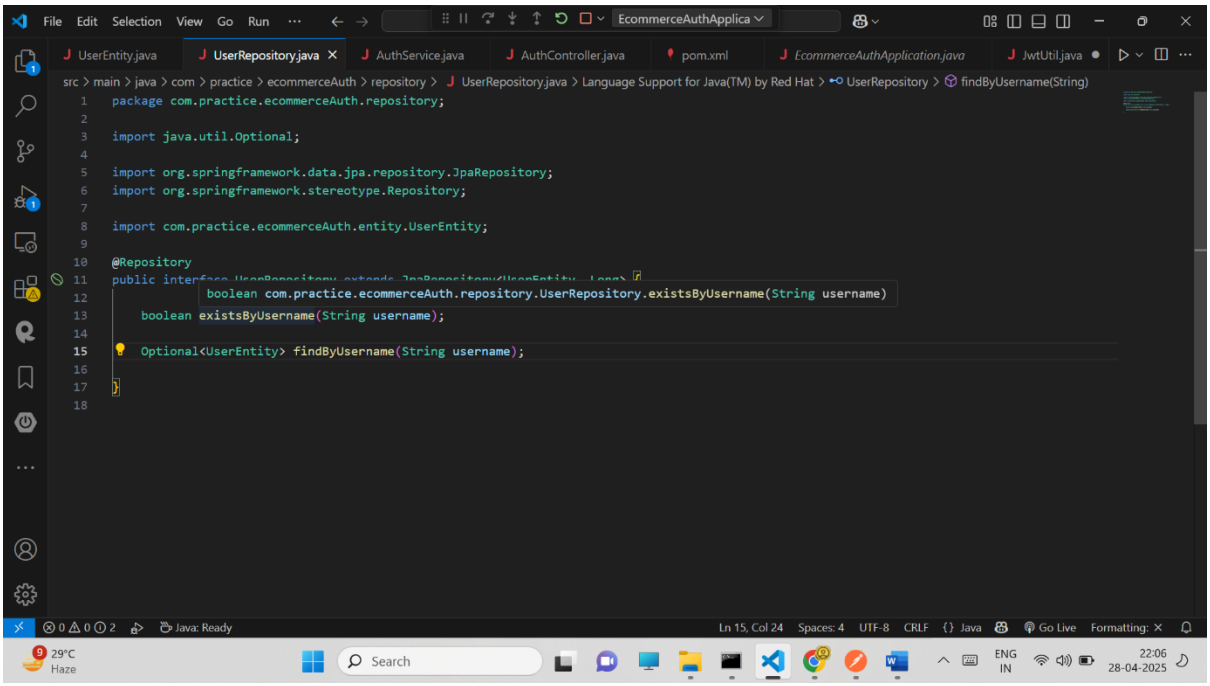
1. UserEntity



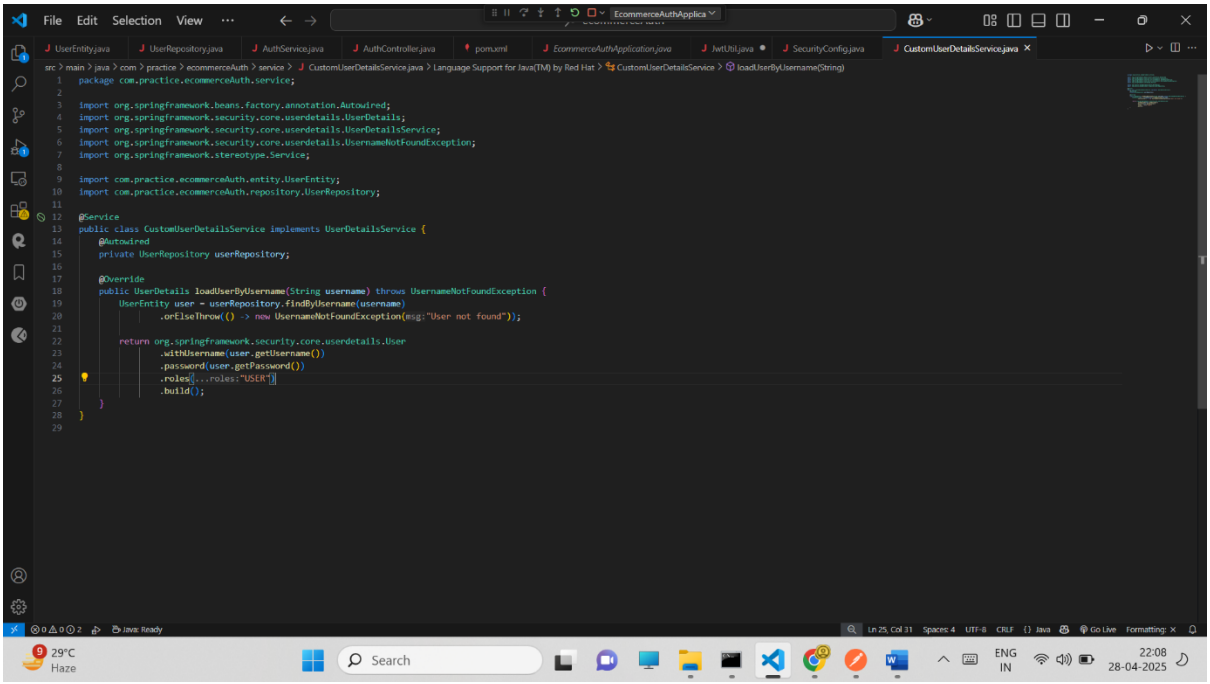
```java
package com.practice.ecommerceAuth.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class UserEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;

    private String username;
    private String password;

    private String role = "USER_ROLE";
}
```

2. UserRepository



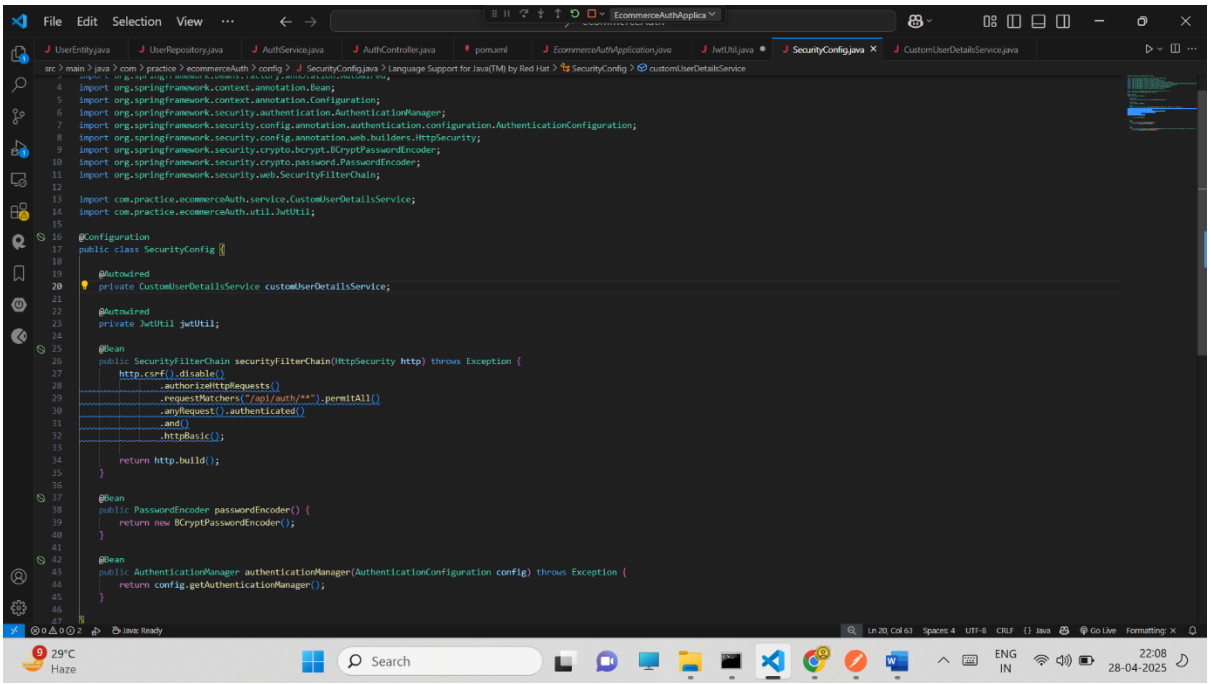
```java
package com.practice.ecommerceAuth.repository;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.practice.ecommerceAuth.entity.UserEntity;

@Repository
public interface UserRepository extends JpaRepository<UserEntity, Long> {
    boolean existsByUsername(String username);

    Optional<UserEntity> findByUsername(String username);

}
```

3. CustomeUserDetailsService



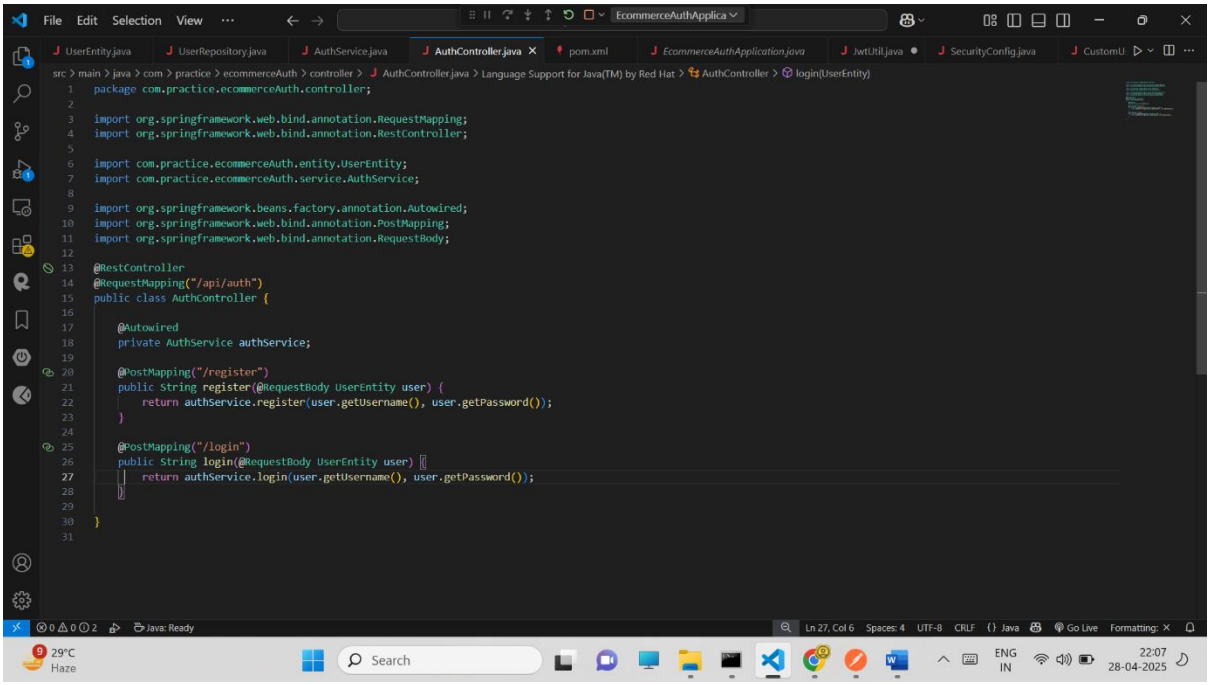
```java
package com.practice.ecommerceAuth.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import com.practice.ecommerceAuth.entity.UserEntity;
import com.practice.ecommerceAuth.repository.UserRepository;

@Service
public class CustomUserDetailsService implements UserDetailsService {
    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        UserEntity user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException(msg:"User not found"));

        return org.springframework.security.core.userdetails.User
                .withUsername(user.getUsername())
                .password(user.getPassword())
                .roles(...roles:"USER")
                .build();
    }
}
```

4. SecurityConfig



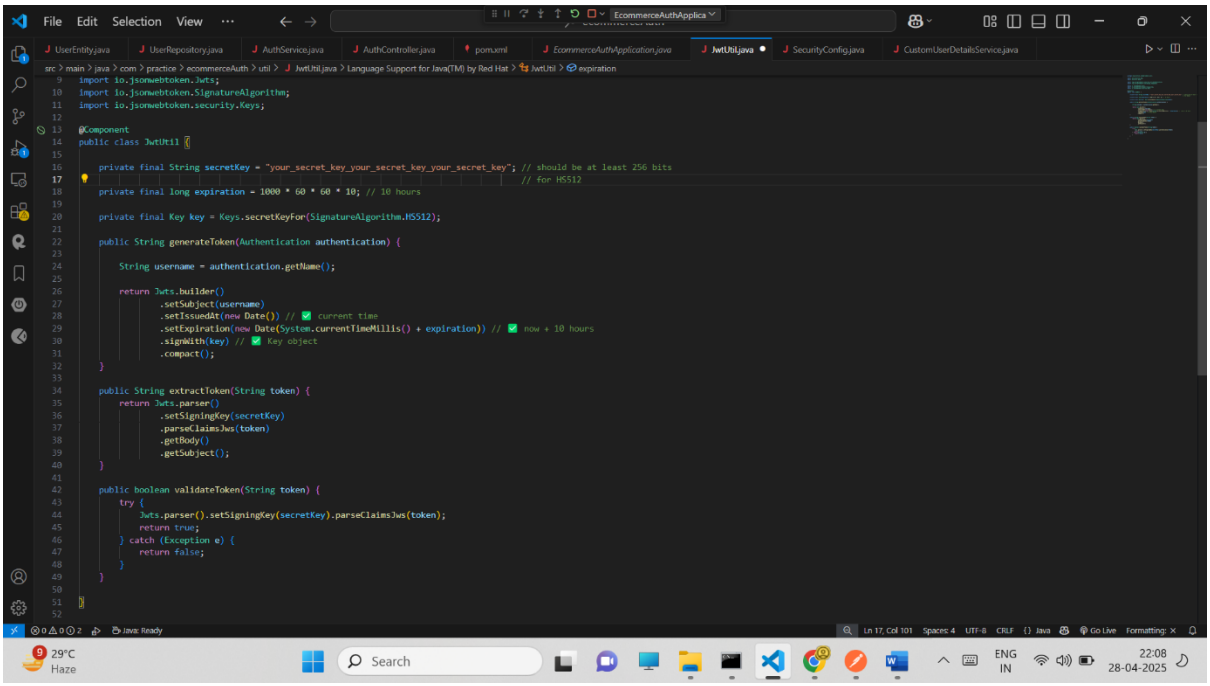
```java
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

import com.practice.ecommerceAuth.service.CustomUserDetailsService;
import com.practice.ecommerceAuth.util.JwtUtil;

@Configuration
public class SecurityConfig {

    @Autowired
    private CustomUserDetailsService customUserDetailsService;

    @Autowired
    private JwtUtil jwtUtil;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http.csrf().disable()
                .authorizeHttpRequests()
                .requestMatchers("/api/auth/**").permitAll()
                .anyRequest().authenticated()
                .and()
                .httpBasic();

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }
}
```

5. AuthController



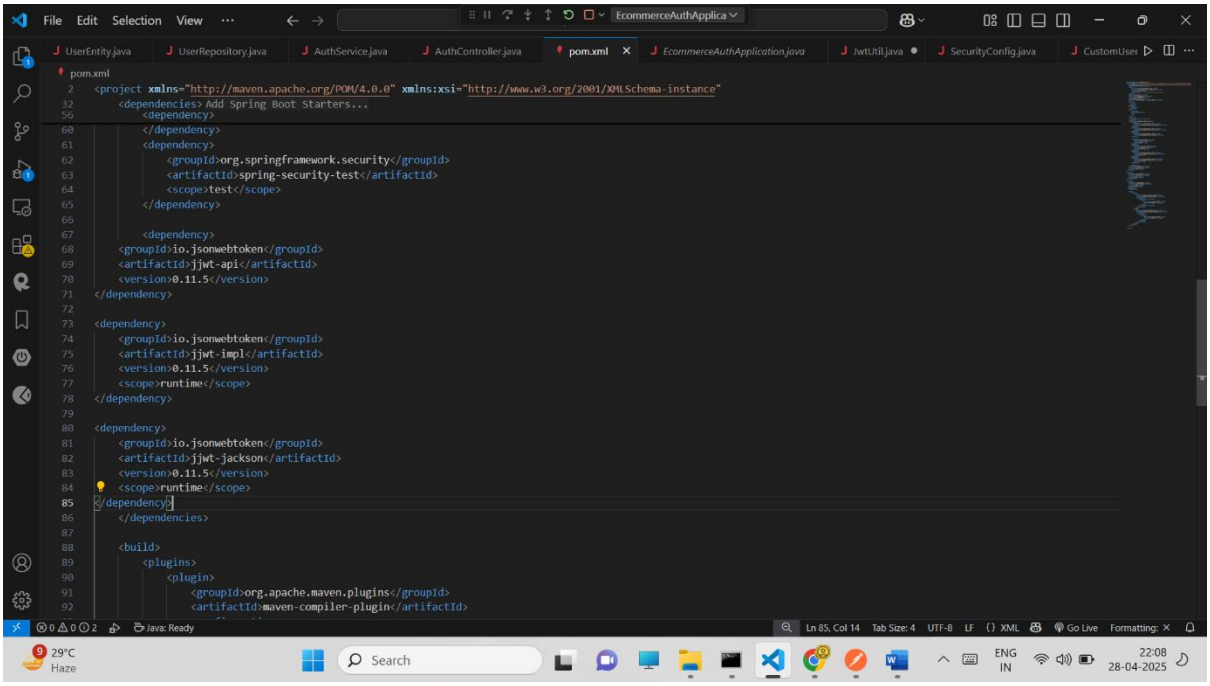
```java
package com.practice.ecommerceAuth.controller;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.practice.ecommerceAuth.entity.UserEntity;
import com.practice.ecommerceAuth.service.AuthService;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthService authService;

    @PostMapping("/register")
    public String register(@RequestBody UserEntity user) {
        return authService.register(user.getUsername(), user.getPassword());
    }

    @PostMapping("/login")
    public String login(@RequestBody UserEntity user) {
        return authService.login(user.getUsername(), user.getPassword());
    }

}
```

6. JwtUtil



```java
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;

@Component
public class JwtUtil {

    private final String secretKey = "your_secret_key_your_secret_key_your_secret_key"; // should be at least 256 bits
                                                                                         // for HS512
    private final long expiration = 1000 * 60 * 60 * 10; // 10 hours

    private final Key key = Keys.secretKeyFor(SignatureAlgorithm.HS512);

    public String generateToken(Authentication authentication) {

        String username = authentication.getName();

        return Jwts.builder()
                .setSubject(username)
                .setIssuedAt(new Date()) // ✅ current time
                .setExpiration(new Date(System.currentTimeMillis() + expiration)) // ✅ now + 10 hours
                .signWith(key) // ✅ Key object
                .compact();
    }

    public String extractToken(String token) {
        return Jwts.parser()
                .setSigningKey(secretKey)
                .parseClaimsJws(token)
                .getBody()
                .getSubject();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parser().setSigningKey(secretKey).parseClaimsJws(token);
            return true;
        } catch (Exception e) {
            return false;
        }
    }

}
```

7. Pom.xml



8. AuthService